C48 — Hướng dẫn tự code các thay đổi SOS

Google Sign-in, xác thực email OTP, tiếp nhận nhanh, AI advisory và phân công cứu hộ

23 tháng 10 năm 2026

Tài liệu hướng dẫn tự triển khai trên repository C48

Lượt này chỉ sửa seeder; các chức năng còn lại dành cho bạn tự code.

Nội dung

- Mục đích
- Tóm tắt hiện trạng
- Nguyên tắc project
- Bản đồ file
- 1. Đăng nhập bằng Google
- 2. Đăng ký email và OTP
- 3. Form SOS: cứu hộ, cứu trợ, người dễ bị tổn thương
- 4. LLM trích JSON từ mô tả nhanh
- 5. Điểm nguy cơ và hỗ trợ cứu hộ thông minh
- 6. Chọn đội theo tải và khoảng cách
- 7. PUT cập nhật SOS
- 8. Thông tin cho điều phối
- 9. Một đội làm cả cứu hộ và cứu trợ
- 10. Seeder tự viết trong lượt này
- 11. Migration cụ thể: viết cả up/down
- 12. Thứ tự tự code
- 13. Checklist tự nghiệm thu sau khi code
- Tài liệu gốc

Mục đích

Tài liệu này hướng dẫn tự triển khai các thay đổi đã nêu cho C48. Nội dung bám theo mã nguồn và contract hiện có, chỉ rõ file cần chạm, API cần thêm, thứ tự xử lý, cấu trúc dữ liệu, code cốt lõi và lý do của từng quyết định.

Trong lượt này chỉ sửa dữ liệu seeder. Google, OTP, form SOS, AI, PUT cập nhật và giao diện điều phối chưa được triển khai. PDF là tài liệu để bạn tự code các phần đó.

Các mục ghi Đề xuất chưa phải chính sách nghiệp vụ đã phê duyệt. Chốt chúng trước khi biến thành contract cố định.

Tóm tắt hiện trạng

Hạng mục • Đã có • Cần tự triển khai

Đăng ký • Username, mật khẩu, tên hiển thị; cấp CITIZEN ngay • Email, OTP, trạng thái chờ xác minh

Đăng nhập • Username/mật khẩu; phiên WEB và NATIVE do Identity cấp • Google Sign-in và liên kết tài khoản an toàn

SOS • Khách không đăng nhập vẫn gửi được; mô tả, số người, liên hệ, vị trí, cờ nguy hiểm tự khai • Phân biệt cứu hộ/cứu trợ, trẻ em/người cao tuổi, nhập nhanh

Sửa SOS • POST bổ sung có version, idempotency và event lịch sử • PUT cập nhật thông tin người báo được phép sửa

AI • Chưa có LLM trích xuất • Job nền, JSON có schema, coordinator duyệt

Nguy cơ • Điều phối viên chọn P1–P4 • Advisory có giải thích; không tự đổi P1–P4

Gợi ý đội • Lọc năng lực, phạm vi, vị trí còn mới, slot; xếp theo tải gần đây và khoảng cách • Hiển thị lịch sử đội và lý do chi tiết

Lịch sử • Timeline request; endpoint missions hiện chỉ trả mission đang hoạt động • Điều phối xem lịch sử, kết quả và đổi đội

Cùng đội cứu hộ/cứu trợ • Chưa có luồng delivery mission trong Response • Thiết kế sau core; Logistics quản lý kho và handoff

Seeder • 13 tình huống SOS, thời gian tương đối theo lúc seed • Fixture B/C/D để xem tradeoff tải và khoảng cách

Nguyên tắc project

1. Mã nguồn, enum, API và tài liệu kỹ thuật giữ tiếng Anh. Nội dung Web/Mobile, lỗi, thông báo và email cho người dùng phải bằng tiếng Việt.
2. OTP dùng cho đăng ký tài khoản, không khóa đường SOS khẩn cấp. Khách vẫn gửi SOS mà không đăng nhập.
3. AI chỉ trích xuất/gợi ý cho điều phối viên. Không để LLM tự xác minh, tự đặt P1, chọn đội, phát lệnh hoặc đổi trạng thái.
4. Ghi rõ dữ kiện do người dân khai báo; không trình bày thành thông tin đã xác minh.
5. Chỉ dùng Compose chính ở backend/infra/compose.yaml. Không tạo database/Compose/container test riêng.
6. Không reset volume khi code. Nếu sau này cần reset demo, chỉ dùng lệnh có cờ --reset trong tài liệu backend; giữ nguyên c48_minio-data.

Bản đồ file

Việc • File hiện có nên đọc/sửa

Auth • backend/apps/identity/src/auth/auth.dto.ts, auth.controller.ts, auth.service.ts, auth-security.ts, auth.module.ts

Schema/migration Identity • docs/backend/schema/identity.sql; backend/apps/identity/src/database/migrations/; backend/apps/identity/src/database/data-source.ts

Tạo SOS • backend/apps/response/src/requests/request-intake.dto.ts, request-intake.controller.ts, request-intake.service.ts, request-intake.module.ts

Theo dõi/bổ sung • backend/apps/response/src/tracking/tracking.dto.ts, tracking.controller.ts, tracking.service.ts

Điều phối/đội • backend/apps/response/src/operations/request-operations.controller.ts, request-operations.dto.ts, request-operations.queries.service.ts, request-operations.team-candidates.service.ts, request-operations.team-candidates.helpers.ts, request-operations.triage.service.ts

Schema/migration Response • docs/backend/schema/response.sql; backend/apps/response/src/database/migrations/; backend/apps/response/src/database/data-source.ts

Contract • contracts/openapi/.json, backend/scripts/contracts/openapi-details.mjs, docs/backend/07-error-catalog.md

Migration rollback checker • backend/scripts/db/check-migration-down

Backend task order • docs/backend/README.md

Frontend • Repository chưa có source Web/Mobile. Khi tạo UI, đặt theo cấu trúc dự án được chọn, ví dụ web/src/features/auth, web/src/features/requests, mobile/src/features/auth, mobile/src/features/sos. Đây là đường dẫn dự kiến, chưa tồn tại.

Schema SQL và API contract phải cập nhật cùng service. Đọc docs/backend/README.md, project context và Appendix A của technology plan trước khi code backend. Không tạo Python service.

1. Đăng nhập bằng Google

1.1 Luồng

Web/Mobile lấy Google ID token bằng SDK/luồng chính thức, rồi gửi token tới Identity. Server xác minh chữ ký và các claim trước khi tin thông tin bên trong.

```
Google SDK -> POST /api/v1/identity/auth/google
-> server verify signature, aud, iss, exp, email_verified
-> lookup provider + sub
-> create C48 session via existing Identity session code
-> WEB: existing secure cookies/CSRF
-> NATIVE: existing access/refresh token response
```

Không gửi email hoặc Google user ID tự khai thay ID token. Khóa danh tính ổn định là claim sub, không phải email. Nếu ứng dụng chỉ cần đăng nhập C48 thì không lưu Google access/refresh token.

1.2 DTO

Thêm vào backend/apps/identity/src/auth/auth.dto.ts:

```
export class GoogleLoginDto {
@IsString()
@MinLength(20)
@MaxLength(8192)
id_token!: string;

@IsIn(['WEB', 'NATIVE'])
client_kind!: 'WEB' | 'NATIVE';

// Chỉ dùng khi người dùng mới hoàn tất tạo tài khoản lần đầu.
@IsOptional()
@IsString()
@MinLength(3)
@MaxLength(64)
@Matches(/^[A-Za-z0-9._-]+$/)
first_time_username?: string;
}
```

Bổ sung import validator còn thiếu trong file. Tên first_time_username là contract đề xuất; có thể dùng màn hình hoàn tất đăng ký riêng, miễn email được server xác minh.

1.3 Controller

Thêm route auth/google vào auth.controller.ts. Hình dạng dưới đây chỉ ra vị trí và luồng; nhánh NATIVE cần dùng đúng response hiện có trong login(), không tạo format thứ hai.

```
@Post('auth/google')
@HttpCode(HttpStatus.OK)
@UseGuards(AuthTransportGuard)
async googleLogin(
@Body() dto: GoogleLoginDto,
@Req() request: Request,
@Res({ passthrough: true }) response: Response,
): Promise<Record<string, unknown>> {
cachePrivate(response);
const session = await this.authService.loginWithGoogle(dto, requestIp(request));

if (dto.client_kind === 'WEB') {
setSessionCookies(
response, this.config, session.accessToken, session.refreshToken,
session.accessExpiresIn, session.absoluteExpiresAt,
);
return {
user: session.user,
session: { expires_at: session.absoluteExpiresAt.toISOString() },
must_change_password: session.mustChangePassword,
};
}

return {
user: session.user,
access_token: session.accessToken,
refresh_token: session.refreshToken,
expires_in: session.accessExpiresIn,
must_change_password: session.mustChangePassword,
};
}
```

Trong code thật, nếu native response hiện chưa được tách thành helper thì giữ nguyên đoạn trả kết quả từ login() hoặc refactor helper dùng chung. Giữ Cache-Control: no-store, transport guard, cookie, CSRF và refresh flow hiện có.

1.4 Service, provider và liên kết

Tạo interface riêng, ví dụ GoogleIdentityVerifier, thay vì trộn lời gọi mạng với SQL:

```
type VerifiedGoogleIdentity = {
subject: string; // claim sub
email: string;
emailVerified: true;
displayName: string;
};

interface GoogleIdentityVerifier {
verify(idToken: string): Promise<VerifiedGoogleIdentity>;
}
```

loginWithGoogle() thực hiện theo thứ tự:

1. Xác minh ID token bằng thư viện server-side.
2. Từ chối nếu thiếu sub/email, email_verified không true, sai chữ ký, issuer, audience hoặc hết hạn.
3. Tìm external identity có provider GOOGLE và subject bằng claim sub.
4. Nếu tìm thấy: yêu cầu user ACTIVE rồi dùng cùng code path tạo session như password login.
5. Nếu chưa liên kết nhưng email đã có trên tài khoản C48: không tự nối theo email. Yêu cầu người dùng đăng nhập tài khoản cũ rồi dùng route liên kết Google có session hợp lệ và xác thực gần đây. Có thể dùng quy trình khôi phục mật khẩu nếu cần.

6. Nếu là người dùng mới: yêu cầu username hợp lệ; tạo user, đánh dấu email Google đã xác minh, tạo liên kết provider và cấp đúng CITIZEN trong một transaction.
7. Tạo C48 session qua session_family, refresh_session và issueSession() hiện có.

Unique constraint trên (provider, subject) ngăn hai login đồng thời tạo duplicate. Nếu race gặp unique violation, đọc lại identity rồi login cùng user.

1.5 Schema và cấu hình

Cập nhật docs/backend/schema/identity.sql và migration:

- Thêm email chuẩn hóa và email_verified_at vào app_user.
- Tạo external_identity có provider, subject, user_id, created_at; unique (provider, subject).
- Xử lý email trùng trước khi tạo unique index.
- Tài khoản Google không có mật khẩu cần quyết định schema. Phương án rõ ràng là cho password_hash nullable, sửa login để dùng dummy hash khi null và vẫn báo lỗi credential chung; hoặc buộc người dùng đặt mật khẩu trước khi tạo account. Không được bỏ qua kiểm tra password.
- Thêm allowlist client ID ở server, ví dụ GOOGLE_CLIENT_IDS. Client ID không phải secret. Không nhúng Google client secret vào browser/app.

Tài liệu chính thức: Xác minh ID token ở backend — https://developers.google.com/identity/sign-in/web/backend-auth, thực hành tốt về Google Sign-in — https://developers.google.com/identity/siwg/best-practices. Chỉ cần authorization-code flow nếu sau này cần quyền Google API; xem hướng dẫn code model — https://developers.google.com/identity/oauth2/web/guides/use-code-model?hl=en.

1.6 Web/Mobile

- Web dùng Google Identity Services trên origin đã đăng ký. Không nhúng luồng OAuth vào WebView.
- Android/iOS lấy token bằng SDK/credential chính thức của nền tảng. Backend xác minh audience thuộc allowlist.
- Email Google đã được verify có thể đáp ứng xác minh email ở lần tạo tài khoản; không cần gửi thêm OTP cho email đó.
- Hiển thị tiếng Việt: “Tiếp tục với Google”, “Không thể xác minh tài khoản Google”, “Đăng nhập tài khoản hiện có để liên kết”.

2. Đăng ký email và OTP

2.1 Contract đề xuất

Mở rộng DTO đăng ký hiện tại:

```
{
"username": "nguyen_van_a",
"email": "a@example.com",
"password": "mật khẩu",
"display_name": "Nguyễn Văn A"
}
```

Kết quả đăng ký thêm verification_required=true và trạng thái PENDING_EMAIL. Chưa tạo session hoặc quyền CITIZEN cho tới khi xác minh.

```
POST /api/v1/identity/auth/email/verify
{ "challenge_id": "...", "code": "123456" }

POST /api/v1/identity/auth/email/resend
{ "email": "a@example.com" }
```

Phản hồi resend nên chung chung: “Nếu thông tin phù hợp, mã xác minh sẽ được gửi.” Không tiết lộ email có tài khoản hay không.

2.2 Schema và quy tắc bảo mật

Hiện RegisterDto chỉ có username/password/display_name, app_user.status chỉ có ACTIVE/DISABLED, và register cấp CITIZEN ngay. Cần thay đổi cả database lẫn service.

- Thêm email và email_verified_at; thêm trạng thái PENDING_EMAIL.
- Tạo email_verification_challenge: id, user_id, email, code_mac, expires_at, attempt_count, consumed_at, created_at, sent_at.
- Tạo OTP bằng crypto.randomInt(0, 1_000_000), pad đủ 6 chữ số.
- Lưu HMAC của mã, không lưu mã thô. HMAC-SHA-256 cần secret riêng từ env; hash thường không đủ cho OTP chỉ có một triệu giá trị.
- Verify trong transaction: khóa challenge, kiểm tra hạn/dùng lại/số lần, đánh dấu consumed, đặt user ACTIVE, cập nhật email_verified_at, cấp CITIZEN đúng một lần.
- Resend vô hiệu mã cũ trước khi tạo challenge mới; rate-limit theo IP và email/user.
- Không log OTP, token hoặc email đầy đủ. Gửi email tiếng Việt qua adapter EmailSender sau khi tạo challenge thành công.
- Provider gửi email chưa được project chọn; dùng interface và cấu hình env, không commit secret.

Mặc định có thể bắt đầu với OTP hết hạn sau 5 phút, tối đa 5 lần nhập, resend cách ít nhất 60 giây và giới hạn theo giờ. Đây là đề xuất cần chốt chính sách.

2.3 File cần sửa

1. auth.dto.ts: email, VerifyEmailDto và ResendEmailOtpDto.
2. auth.controller.ts: verify/resend routes, transport guard, no-store và idempotency phù hợp.
3. auth.service.ts: register tạo pending; verify dùng transaction/row lock; không cấp role trong register.
4. auth.module.ts: đăng ký EmailSender, Google verifier và config.
5. identity.sql/migration: email, status, OTP table, unique index, foreign key.
6. .env.example: chỉ thêm tên biến, không có secret thật.
7. OpenAPI/error catalog: thêm field và mã lỗi ổn định như EMAIL_OTP_INVALID, EMAIL_OTP_EXPIRED, EMAIL_OTP_RATE_LIMITED; message backend bằng tiếng Việt.

8. Auth UI: nhập email, màn hình mã 6 số, gửi lại, đếm giờ, sửa email, paste OTP và accessibility label tiếng Việt.

Không bắt xác minh email trước khi gửi SOS. Guest intake và tracking secret vẫn độc lập.

OWASP khuyến nghị token xác minh đủ ngẫu nhiên, có hạn, dùng một lần và có rate-limit: Email Validation and Verification Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Email_Validation_and_Verification_Cheat_Sheet.html.

3. Form SOS: cứu hộ, cứu trợ, người dễ bị tổn thương

3.1 Tái sử dụng trường nguy hiểm

CreateSelfRequestDto đã có reporter_declared_danger. Dùng field này cho câu hỏi tiếng Việt "Có nguy hiểm đến tính mạng?". Coordinator phải thấy đây là "Người báo tự khai nguy hiểm", chưa phải kết luận đã xác minh.

3.2 Field mới và validation

Thêm vào request DTO (proxy DTO đang kế thừa self DTO):

```
@IsOptional()
@IsBoolean()
needs_rescue?: boolean; // mặc định true

@IsOptional()
@IsBoolean()
needs_relief_supplies?: boolean; // mặc định false

@IsOptional()
@IsInt()
children_count?: number | null;

@IsOptional()
@IsInt()
elderly_count?: number | null;
```

Service áp dụng quy tắc:

- Nếu cả nhu cầu cứu hộ và cứu trợ đều false thì validation error tiếng Việt.
- Client cũ không gửi cờ nhu cầu: mặc định rescue=true, supplies=false.
- Chưa biết số lượng thì null/không gửi; không tự biến thành 0.
- Đếm trong khoảng 0..10.000; không lớn hơn people_affected nếu tổng đã biết.
- Nếu tổng và cả hai nhóm tuổi đều biết, tổng trẻ em + người cao tuổi không vượt quá tổng người.
- Không yêu cầu công dân nhập SKU/hàng hóa; coordinator/Logistics tạo ReliefNeed chuẩn sau xác minh.

Thêm children_count và elderly_count vào request_subject; thêm needs_rescue và needs_relief_supplies vào assistance_request. Migration cần default tương thích dữ liệu cũ và CHECK ít nhất một cờ true. Cập nhật approved-facts snapshot, tracking view, supplement và coordinator detail. DTO thay đổi mà không cập nhật snapshot sẽ làm điều phối thiếu facts được xác minh.

3.3 Nhãn form

- "Mô tả nhanh": "Nói ngắn gọn chuyện gì đang xảy ra, có bao nhiêu người và cần giúp gì."
- Nhu cầu: "Cần cứu hộ" mặc định bật; "Cần cứu trợ hàng hóa" có thể bật thêm. Cho phép chọn cả hai.
- "Có nguy hiểm đến tính mạng?"
- Tổng số người, số trẻ em, số người cao tuổi.
- GPS/ghim vị trí; tên và số điện thoại nếu cung cấp.
- Với báo hộ: quan hệ, nguồn biết tin, lần cuối liên lạc, khả năng liên lạc.

Nút gửi SOS không chờ OTP hoặc AI.

4. LLM trích JSON từ mô tả nhanh

4.1 Luồng nền

Dùng description hiện có làm đoạn nhập tự do. Khi request đã lưu/commit, tạo job trích xuất nền bên trong Response; không tạo Python service. Trả tracking ngay cả khi LLM lỗi/chậm.

```
POST SOS -> lưu request + event -> trả tracking
|
v
job nền Response
|
v
validate JSON schema
|
v
lưu advisory STAFF-only cho coordinator duyệt
```

Mặc định không gửi tên, phone, tracking secret, chính xác GPS hay ảnh ra provider. Chỉ gửi mô tả đã tối thiểu hóa. Provider LLM chưa được chọn; cần quyết định retention/data handling trước khi gửi dữ liệu thật.

4.2 JSON có giới hạn

```
{
"incident_category_code": "FLOOD",
"people_affected": 5,
"children_count": 2,
"elderly_count": 1,
"reporter_declared_danger": true,
"needs_rescue": true,
"needs_relief_supplies": false,
"risk_signals": ["TRAPPED_BY_WATER", "CHILDREN_PRESENT"],
"missing_fields": ["WATER_LEVEL_TREND"],
"conflicts": [],
"field_evidence": {
"children_count": "nhà có hai bé nhỏ",
"risk_signals": "nước vào nhà, chưa ra ngoài được"
}
}
```

Schema runtime phải giới hạn enum, số, độ dài list/text và từ chối additionalProperties. Không bịa số hoặc kết luận nguy hiểm nếu nguồn không nói. Dữ liệu không chắc là null/unknown.

Tạo AiAdvisoryProvider và AiExtractionWorker trong Response. Advisory lưu request_id, input hash, model/provider, prompt_version, schema_version, status, JSON, created_at. Không đưa AI advisory vào timeline công khai. Coordinator xem câu trích gốc, chấp nhận/sửa/bỏ qua; mọi thay đổi facts chính thức phải qua command và lưu event.

LLM không có tool, quyền DB hay quyền gọi dispatch. Coi nội dung công dân là dữ liệu, không phải chỉ thị. JSON sai, timeout hoặc prompt injection không được đổi status/priority/team. Tham khảo OWASP LLM Prompt Injection Prevention — https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html.

5. Điểm nguy cơ và hỗ trợ cứu hộ thông minh

5.1 Tách advisory khỏi quyết định

- P1–P4 là mức chính thức do coordinator; hiện có triage service.
- Risk advisory xếp việc cần xem trước, có lý do/nguồn.
- Candidate service chỉ gợi ý đội đủ điều kiện; coordinator vẫn chọn.

Không để LLM tự xác minh hoặc dispatch. Không dùng điểm tổng để phát lệnh.

5.2 Rule ban đầu dễ giải thích

Tín hiệu • Gợi ý

Nước đang lên + người bị kẹt/không tự đi được • Cảnh báo khẩn, liên hệ xác minh ngay

Có trẻ em/người cao tuổi + không sơ tán được • Tăng chú ý, hỏi khả năng di chuyển/liên lạc

Bị thương nặng/khó thở • Đánh dấu cần xác minh y tế khẩn

Chỉ xin hàng cứu trợ • Đưa vào luồng nhu cầu cứu trợ

Thiếu/mâu thuẫn dữ liệu hoặc chỉ có AI suy đoán • Cần coordinator xem, không tự nâng P

Lưu stable reason code như FLOOD_RISING_AND_TRAPPED, VULNERABLE_PERSONS_NO_EVACUATION, REPORT_CONFLICT rồi map sang tiếng Việt. Reporter_declared_danger=true làm tín hiệu kiểm tra nhanh, không tự biến thành P1. Thiếu dữ liệu không được dùng để phạt người báo.

Ban đầu chạy advisory shadow: lưu đề xuất, chưa tác động queue chính. Coordinator accept/adjust/reject có lý do. Trước khi dùng score thật, đánh giá false negative và chính sách với con người.

6. Chọn đội theo tải và khoảng cách

6.1 Hiện trạng cần giữ đúng

Candidate service hiện lọc năng lực, phạm vi, vị trí mới và slot; bán kính mặc định 10 km. Tolerance hiện tại: P1 0 m, P2 500 m, P3/P4 2 km. Trong dải ưu tiên, xếp tải mission gần đây rồi khoảng cách. Xem request-operations.team-candidates.service.ts và request-operations.team-candidates.helpers.ts trước khi sửa.

6.2 Hai ví dụ

Với P3, B cách 3 km nhưng nhiều việc; C cách 4 km và ít việc; D cách 10 km, ít việc:

- nearest=3 km; tolerance=2 km, nên B/C cùng vào dải khoảng 5 km. Nếu C ít tải hơn, gợi ý C.
- D ở 10 km ngoài dải ưu tiên, không thắng chỉ vì rảnh. Nếu B/C không đủ năng lực thì coordinator có thể xem các đội xa hơn theo chính sách.
- Như vậy cân bằng tải trong nhóm đủ gần, không ưu tiên một đội quá xa.

6.3 Thuật toán

```
eligible = skill match
AND scope match
AND fresh position
AND no active mission

nearest = minimum eligible distance
preferred = eligible where distance <= min(maxRadius, nearest + tolerance(priority))

sort by:
recentMissionCount24h ASC
distanceMeters ASC
lastAssignedAt ASC
teamId ASC
```

TeamId là tie-break cuối để kết quả deterministic. Candidate response nên có khoảng cách, tải 24h, năng lực, vị trí/thời điểm, lý do shortlist và dữ kiện coordinator cần biết. Lưu suggested_team_id, override_reason và lịch sử mission.

Về sau có thể thay số mission bằng effort score: mission on-scene kéo dài tính nặng hơn; failed mission vẫn tính vì đội đã bỏ công; offered có thể tính nhẹ hơn. Đây là đề xuất chính sách, chưa có trong thuật toán hiện tại.

6.4 Lịch sử đội

Không xóa event khi đổi đội. Ghi mission lifecycle, thời gian, outcome, coordinator, reason và provenance RADIO/PHONE/IN_PERSON khi điều phối ghi thay đội. Hiện GET /api/v1/response/requests/:id/missions chỉ trả mission đang hoạt động. Mở rộng query có phân trang để xem lịch sử/kết quả; không trả toàn bộ vô giới hạn.

7. PUT cập nhật SOS

Đã có POST /api/v1/response/requests/:id/supplements. TrackingService.supplement() kiểm tra ReporterAccess, expected_version, lưu event bất biến và xử lý khác nhau trước/sau verify. Đọc luồng đó trước.

Đề xuất thêm PUT /api/v1/response/requests/:id/report cho thông tin người báo sở hữu:

```
{
"expected_version": 4,
"description": "Nước đã lên tới bậc cửa, trong nhà có 5 người.",
"people_affected": 5,
"children_count": 2,
"elderly_count": 1,
"reporter_declared_danger": true,
"needs_rescue": true,
"needs_relief_supplies": true
}
```

Không cho PUT status, priority, verification, team, vùng đã duyệt hoặc Logistics data. Cần:

1. Kiểm tra quyền bằng ReporterAccessService, tracking code/secret hoặc phiên công dân.
2. So expected_version; stale version trả conflict tiếng Việt.
3. Allowlist field; field lạ bị từ chối.
4. Transaction cập nhật field được phép + tăng version.
5. Ghi request_event bất biến gồm actor, before/after, visibility và thời điểm.
6. Nếu request đã verify/attribution-locked thì tạo supplement chờ review, không sửa facts đã duyệt.
7. Idempotency phù hợp; cùng key/body trả lại kết quả cũ.

Quyết định rõ semantics: đề xuất field thiếu nghĩa là giữ nguyên; explicit null xóa field nếu được phép. Ghi vào OpenAPI để không hiểu PUT là thay toàn bộ resource. PUT không tự đổi priority hay đội.

8. Thông tin cho điều phối

Trang detail nên có:

1. SOS: mã, giờ nhận, status, priority chính thức, vùng/campaign, version.
2. Người gửi/liên hệ: SELF/PROXY, tên/số liên hệ, người thay thế, trạng thái liên hệ; PII chỉ hiển thị theo scope.
3. Tình huống: mô tả nguyên văn, vị trí/độ chính xác/thời điểm GPS, tổng người, trẻ em, người cao tuổi, nguy hiểm tự khai.
4. Nhu cầu cứu hộ/cứu trợ, Logistics ReliefNeed và tiến độ cấp phát.
5. Xác minh: người xác minh, kết quả, facts đã duyệt, evidence, thời gian.
6. AI advisory: nguồn, JSON, model/prompt version, missing/conflicting facts, coordinator action.
7. Đội: candidates, distance, workload, skills, lý do, override, lịch sử mission.
8. Timeline: submit, supplement, contact, verify, triage, offer, accept/decline, en-route, on-scene, outcome, handoff.

Người gửi chỉ xem event REPORTER và nội dung công khai. Biết tracking_code không đủ để lấy PII. Mọi nhãn trạng thái hiển thị bằng tiếng Việt.

9. Một đội làm cả cứu hộ và cứu trợ

Không tạo bản sao rescue team. Dùng một rescue_team; phân loại nhiệm vụ bằng mission kind/intention.

Hiện DB giới hạn một mission hoạt động mỗi đội bằng unique partial index mission_team_one_active_uq. Đề xuất:

- Giai đoạn đầu giữ một việc hoạt động/đội; mission kind là RESCUE hoặc DELIVERY, đội làm tuần tự.
- Chỉ mở song song khi đã chốt capacity model và transaction/locking; không chỉ xóa unique index.
- Delivery mission tham chiếu distribution ID opaque của Logistics.
- Logistics vẫn sở hữu approve, reserve, stock ledger, dispatch và handoff. Response không tự trừ kho; chỉ handoff event từ Logistics xác nhận đã giao.

Delivery-carrier là hạng mục tùy chọn sau core trong plan. Làm sau SOS/mission và Logistics handoff.

10. Seeder tự viết trong lượt này

Đã chỉnh docs/backend/seed/seed-response.sql:

- Seed cũ có 13 request tình huống và timestamp tương đối theo now(); campaign seed đã có mốc tương lai now()+30 days.
- Bổ sung đội demo B/C/D dùng reporting mode COORDINATOR và vị trí do coordinator ghi qua radio.
- R13 là request P3 chưa có mission, làm mốc khoảng cách.
- B cách khoảng 3 km và có 2 mission 24 giờ gần đây: một FAILED, một COMPLETED.
- C cách khoảng 4 km và không có mission; D cách khoảng 10 km và không có mission.
- Bổ sung assertions trong SQL kiểm tra số đội, distance, workload, provenance RADIO.
- Tất cả timestamp fixture dùng now() +/− interval để mỗi lần seed mới có timeline tương đối. Không tạo SOS ở thời gian tương lai; request cứu nạn tương lai sẽ là dữ liệu sai, còn campaign/hoạt động lên lịch có thể nằm ở tương lai.
- Seed cố ý dùng ID ổn định và ON CONFLICT DO NOTHING. DB đã seed trước đó có thể chưa hiện hàng mới vì seed không update ID cũ.

Fixture giúp minh họa khi C nằm trong dải ưu tiên thì C rảnh có thể được gợi ý; D quá xa không thắng chỉ vì rảnh. Đây là static SQL change; tôi không chạy seed, không reset database và không xác nhận SQL đã chạy. Không xóa volume thủ công. Nếu sau này cần nạp lại demo, theo backend README và chỉ dùng lệnh reset có cờ --reset; giữ MinIO volume.

11. Migration cụ thể: viết cả up/down

Phần này là migration TypeORM mẫu để bạn đưa thành file thật. Timestamp là ví dụ theo ngày của tài liệu; khi code hãy chọn số duy nhất lớn hơn migration mới nhất. Repository đăng ký migration bằng danh sách import tường minh trong data-source.ts.

11.1 Quy tắc migration của C48

- Schema chuẩn là docs/backend/schema/identity.sql và response.sql.
- Initial migration gọi applySchema() và đọc schema chuẩn hiện tại. Database mới có thể đã có cột/bảng mới trước khi delta migration chạy.
- Delta migration vì vậy dùng ADD COLUMN IF NOT EXISTS, CREATE TABLE/INDEX IF NOT EXISTS và constraint có tên ổn định. Cách này vừa nâng database cũ, vừa chạy an toàn sau Initial trên database mới.
- Không sửa InitialIdentity1710000000000 hoặc InitialResponse1710000000000. Thêm migration, sửa schema chuẩn và import class vào data-source.ts.
- Bảng mới phải cấp quyền cho role ứng dụng của service; GRANT SELECT ON ALL TABLES không tự áp dụng hồi tố trên database cũ.
- Down phải từ chối nếu sẽ làm mất dữ liệu. Mỗi service chỉ migrate database của chính nó; không FK qua database service khác.

11.2 Identity: email, OTP và Google identity

Tạo file:

```
backend/apps/identity/src/database/migrations/AddAuthEmailOtpGoogle1792713600000.ts
```

```
import { MigrationInterface, QueryRunner } from 'typeorm';

export class AddAuthEmailOtpGoogle1792713600000 implements MigrationInterface {
async up(q: QueryRunner): Promise<void> {
const statements = [
"ALTER TABLE app_user ADD COLUMN IF NOT EXISTS email text",
"ALTER TABLE app_user ADD COLUMN IF NOT EXISTS email_verified_at timestamptz",
"ALTER TABLE app_user ALTER COLUMN password_hash DROP NOT NULL",
"ALTER TABLE app_user DROP CONSTRAINT IF EXISTS app_user_status_check",
"ALTER TABLE app_user ADD CONSTRAINT app_user_status_check CHECK (status IN
('ACTIVE','DISABLED','PENDING_EMAIL'))",
"CREATE UNIQUE INDEX IF NOT EXISTS app_user_email_lower_uq ON app_user (lower(email)) WHERE email
IS NOT NULL",
"CREATE TABLE IF NOT EXISTS external_identity (provider text NOT NULL CHECK (provider IN
('GOOGLE')), subject text NOT NULL CHECK (btrim(subject) <> ''), user_id uuid NOT NULL REFERENCES
app_user(id), created_at timestamptz NOT NULL DEFAULT now(), PRIMARY KEY (provider, subject),
UNIQUE (user_id, provider))",
"CREATE TABLE IF NOT EXISTS email_verification_challenge (id uuid PRIMARY KEY DEFAULT
gen_random_uuid(), user_id uuid NOT NULL REFERENCES app_user(id), email text NOT NULL, code_mac
text NOT NULL CHECK (code_mac ~ '^[0-9a-f]{64}$'), attempt_count int NOT NULL DEFAULT 0 CHECK
(attempt_count BETWEEN 0 AND 5), created_at timestamptz NOT NULL DEFAULT now(), expires_at
timestamptz NOT NULL, consumed_at timestamptz, invalidated_at timestamptz, sent_at timestamptz,
CHECK (expires_at > created_at))",
"CREATE INDEX IF NOT EXISTS email_verification_open_idx ON email_verification_challenge (user_id,
created_at DESC, id DESC) WHERE consumed_at IS NULL AND invalidated_at IS NULL",
"GRANT SELECT, INSERT ON external_identity TO c48_identity_app",
"GRANT SELECT, INSERT, UPDATE, DELETE ON email_verification_challenge TO c48_identity_app",
];
for (const statement of statements) await q.query(statement);
}

async down(q: QueryRunner): Promise<void> {
await q.query("DO $rollback_guard$ BEGIN IF EXISTS (SELECT 1 FROM external_identity) OR EXISTS
(SELECT 1 FROM email_verification_challenge) OR EXISTS (SELECT 1 FROM app_user WHERE email IS NOT
NULL OR password_hash IS NULL OR status = 'PENDING_EMAIL') THEN RAISE EXCEPTION 'Refusing Identity
rollback: email/Google data exists'; END IF; END; $rollback_guard$");
const statements = [
"DROP TABLE IF EXISTS email_verification_challenge",
"DROP TABLE IF EXISTS external_identity",
"DROP INDEX IF EXISTS app_user_email_lower_uq",
"ALTER TABLE app_user DROP COLUMN IF EXISTS email_verified_at",
"ALTER TABLE app_user DROP COLUMN IF EXISTS email",
"ALTER TABLE app_user ALTER COLUMN password_hash SET NOT NULL",
"ALTER TABLE app_user DROP CONSTRAINT IF EXISTS app_user_status_check",
"ALTER TABLE app_user ADD CONSTRAINT app_user_status_check CHECK (status IN
('ACTIVE','DISABLED'))",
];
for (const statement of statements) await q.query(statement);
}
}
```

Giải thích và service đi kèm:

- email nullable để giữ account cũ chưa có email; RegisterDto mới vẫn bắt buộc email.
- Unique lower(email) chặn email khác kiểu viết hoa/thường tạo hai tài khoản.
- password_hash nullable chỉ cho Google-only account. Sửa login(): password_hash null thì vẫn verify dummy hash, rồi trả INVALID_CREDENTIALS như account không tồn tại/disabled/pending.
- external_identity khóa bằng cặp GOOGLE + sub đã verify; không tự nối theo email.
- Challenge chỉ lưu HMAC; không lưu OTP gốc. Hạn dùng, attempt count và single-use do service kiểm soát.

• Down từ chối nếu có email/Google/pending mới để không âm thầm mất dữ liệu.
• attempt_count=5 là ngưỡng mẫu; nếu chính sách đổi, cập nhật migration, service và thông báo cùng nhau.

Cập nhật docs/backend/schema/identity.sql: app_user có password_hash nullable, email, email_verified_at và status CHECK gồm ACTIVE/DISABLED/PENDING_EMAIL; thêm bảng/index giống migration; thêm GRANT cuối file. User bootstrap vẫn ACTIVE và có password hash; tài khoản cũ có email NULL.

Các thay đổi service:

1. RegisterDto thêm email; trim/lowercase, validate và đưa email chuẩn hóa vào idempotency request hash.
2. register() tạo PENDING_EMAIL và challenge trong transaction; chưa tạo session/CITIZEN.
3. Sau commit, EmailSender gửi mã tiếng Việt. Nếu provider lỗi, account còn pending và có thể resend; không trả mã qua HTTP.
4. verify() khóa challenge/user bằng SELECT FOR UPDATE; kiểm tra code, hạn, attempt count và dùng một lần; sau đó consume, đặt ACTIVE/email_verified_at và cấp CITIZEN đúng một lần.
5. resend vô hiệu challenge cũ rồi tạo challenge mới; rate-limit theo IP và user/email.
6. Google login xác minh ID token trước; lookup (provider, sub). User mới được tạo ACTIVE, email_verified_at, external_identity và CITIZEN cùng transaction. Email trùng account hiện có phải đăng nhập account đó để liên kết.
7. Thêm EMAIL_OTP_PEPPER, GOOGLE_CLIENT_IDS vào config validation và .env.example; không commit secret.

HMAC helper mẫu:

```
import { createHmac, randomInt, timingSafeEqual } from 'node:crypto';

export function createEmailOtp(pepper: string, challengeId: string) {
const code = randomInt(0, 1_000_000).toString().padStart(6, '0');
const codeMac = createHmac('sha256', pepper)
.update(challengeId + ':' + code)
.digest('hex');
return { code, codeMac };
}

export function emailOtpMatches(
pepper: string,
challengeId: string,
code: string,
storedMac: string,
): boolean {
if (!/^[0-9]{6}$/.test(code) || !/^[0-9a-f]{64}$/.test(storedMac)) return false;
const expected = Buffer.from(storedMac, 'hex');
const actual = createHmac('sha256', pepper)
.update(challengeId + ':' + code)
.digest();
return expected.length === actual.length && timingSafeEqual(expected, actual);
}
```

Chỉ gửi code qua EmailSender, chỉ ghi codeMac xuống DB. Dùng pepper riêng, không tái sử dụng pepper SOS/refresh. Cập nhật auth.dto.ts, auth.controller.ts, auth.service.ts, auth.module.ts, auth config, .env.example, docs/backend/02-identity-api.md, 07-error-catalog.md và contracts/openapi/identity.json.

11.3 Response: facts SOS, mission kết hợp và lịch sử

Tạo file:

```
backend/apps/response/src/database/migrations/AddSosFactsAndMissionKind1792713600000.ts
```

Một đội có thể làm cứu hộ, giao hàng hoặc cả hai trong một mission COMBINED. Giữ capacity hiện tại: một mission đang hoạt động trên mỗi đội. distribution_id là UUID opaque của Logistics, không FK sang database Logistics.

```
import { MigrationInterface, QueryRunner } from 'typeorm';

export class AddSosFactsAndMissionKind1792713600000 implements MigrationInterface {
async up(q: QueryRunner): Promise<void> {
const statements = [
"ALTER TABLE assistance_request ADD COLUMN IF NOT EXISTS needs_rescue boolean NOT NULL DEFAULT
true",
"ALTER TABLE assistance_request ADD COLUMN IF NOT EXISTS needs_relief_supplies boolean NOT NULL
DEFAULT false",
"ALTER TABLE assistance_request DROP CONSTRAINT IF EXISTS assistance_request_needs_check",
"ALTER TABLE assistance_request ADD CONSTRAINT assistance_request_needs_check CHECK (needs_rescue
OR needs_relief_supplies)",
"ALTER TABLE request_subject ADD COLUMN IF NOT EXISTS children_count int CHECK (children_count IS
NULL OR children_count BETWEEN 0 AND 10000)",
"ALTER TABLE request_subject ADD COLUMN IF NOT EXISTS elderly_count int CHECK (elderly_count IS
NULL OR elderly_count BETWEEN 0 AND 10000)",
"ALTER TABLE mission ADD COLUMN IF NOT EXISTS mission_kind text NOT NULL DEFAULT 'RESCUE'",
"ALTER TABLE mission ADD COLUMN IF NOT EXISTS distribution_id uuid",
"ALTER TABLE mission DROP CONSTRAINT IF EXISTS mission_kind_check",
"ALTER TABLE mission ADD CONSTRAINT mission_kind_check CHECK (mission_kind IN
('RESCUE','DELIVERY','COMBINED'))",
"ALTER TABLE mission DROP CONSTRAINT IF EXISTS mission_distribution_shape",
"ALTER TABLE mission ADD CONSTRAINT mission_distribution_shape CHECK ((mission_kind = 'RESCUE' AND
distribution_id IS NULL) OR (mission_kind IN ('DELIVERY','COMBINED') AND distribution_id IS NOT
NULL))",
"CREATE INDEX IF NOT EXISTS mission_request_history_idx ON mission (request_id, created_at DESC, id
DESC)",
"GRANT SELECT, INSERT, UPDATE ON assistance_request, request_subject, mission TO c48_response_app",
];
for (const statement of statements) await q.query(statement);
}

async down(q: QueryRunner): Promise<void> {
await q.query("DO $rollback_guard$ BEGIN IF EXISTS (SELECT 1 FROM assistance_request WHERE
needs_rescue = false OR needs_relief_supplies = true) OR EXISTS (SELECT 1 FROM request_subject
WHERE children_count IS NOT NULL OR elderly_count IS NOT NULL) OR EXISTS (SELECT 1 FROM mission
WHERE mission_kind <> 'RESCUE' OR distribution_id IS NOT NULL) THEN RAISE EXCEPTION 'Refusing
Response rollback: new SOS/mission facts exist'; END IF; END; $rollback_guard$");
const statements = [
"DROP INDEX IF EXISTS mission_request_history_idx",
"ALTER TABLE mission DROP CONSTRAINT IF EXISTS mission_distribution_shape",
"ALTER TABLE mission DROP CONSTRAINT IF EXISTS mission_kind_check",
"ALTER TABLE mission DROP COLUMN IF EXISTS distribution_id",
"ALTER TABLE mission DROP COLUMN IF EXISTS mission_kind",
"ALTER TABLE request_subject DROP COLUMN IF EXISTS elderly_count",
"ALTER TABLE request_subject DROP COLUMN IF EXISTS children_count",
"ALTER TABLE assistance_request DROP CONSTRAINT IF EXISTS assistance_request_needs_check",
"ALTER TABLE assistance_request DROP COLUMN IF EXISTS needs_relief_supplies",
"ALTER TABLE assistance_request DROP COLUMN IF EXISTS needs_rescue",
];
for (const statement of statements) await q.query(statement);
}
}
```

Cập nhật docs/backend/schema/response.sql cùng cấu trúc:

- assistance_request thêm hai cờ default true/false và CHECK ít nhất một cờ true.
- request_subject thêm children_count/elderly_count nullable, mỗi số 0..10.000.

- mission thêm mission_kind DEFAULT RESCUE, distribution_id UUID nullable và hai CHECK như migration.
- Thêm mission_request_history_idx theo (request_id, created_at DESC, id DESC).
- Giữ mission_team_one_active_uq: mission COMBINED chỉ chiếm một slot.
- Quyền SELECT/INSERT/UPDATE của bảng mutable phải tiếp tục đúng.

Service/API đi cùng: request-intake DTO/service, approved-facts snapshot, tracking view, supplement, coordinator detail và missions DTO/command/query. Client cũ mặc định RESCUE. Chỉ tạo DELIVERY/COMBINED khi Logistics xác nhận distribution qua API; Response không đọc DB Logistics trực tiếp. Validation tổng người và số trẻ/người già nằm trong service vì CHECK đơn cột không kiểm tra được tổng ở bảng khác.

11.4 Response: bảng advisory AI

Tạo file:

```
backend/apps/response/src/database/migrations/AddAiAdvisory1792713600001.ts
```

```
import { MigrationInterface, QueryRunner } from 'typeorm';

export class AddAiAdvisory1792713600001 implements MigrationInterface {
async up(q: QueryRunner): Promise<void> {
await q.query("CREATE TABLE IF NOT EXISTS ai_advisory (id uuid PRIMARY KEY DEFAULT
gen_random_uuid(), request_id uuid NOT NULL REFERENCES assistance_request(id), request_version int
NOT NULL CHECK (request_version >= 1), input_hash text NOT NULL CHECK (input_hash ~
'^[0-9a-f]{64}$'), provider_code text NOT NULL CHECK (btrim(provider_code) <> ''), model_code text
NOT NULL CHECK (btrim(model_code) <> ''), prompt_version text NOT NULL CHECK (btrim(prompt_version)
<> ''), schema_version text NOT NULL CHECK (btrim(schema_version) <> ''), status text NOT NULL
CHECK (status IN ('PENDING','READY','FAILED')), result_json jsonb, error_code text, created_at
timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now(), CHECK ((status =
'PENDING' AND result_json IS NULL AND error_code IS NULL) OR (status = 'READY' AND result_json IS
NOT NULL AND jsonb_typeof(result_json) = 'object' AND error_code IS NULL) OR (status = 'FAILED' AND
result_json IS NULL AND error_code IS NOT NULL)))");
await q.query("CREATE INDEX IF NOT EXISTS ai_advisory_request_history_idx ON ai_advisory
(request_id, created_at DESC, id DESC)");
await q.query("GRANT SELECT, INSERT, UPDATE ON ai_advisory TO c48_response_app");
}

async down(q: QueryRunner): Promise<void> {
await q.query("DO $rollback_guard$ BEGIN IF EXISTS (SELECT 1 FROM ai_advisory) THEN RAISE EXCEPTION
'Refusing Response rollback: AI advisory rows exist'; END IF; END; $rollback_guard$");
await q.query("DROP TABLE IF EXISTS ai_advisory");
}
}
```

Thêm bảng/index/grant giống vậy trong docs/backend/schema/response.sql và block quyền mutable. Bảng chỉ lưu JSON advisory đã validate, input hash, model/prompt/schema version và mã lỗi ổn định. Không lưu raw provider error, tên, phone, GPS hoặc prompt PII. Coordinator accept/reject ghi request_event; AI advisory không tự thay priority/mission. Chốt retention và provider trước khi bật job.

11.5 Đăng ký migration và rollback checker

Identity data source hiện liệt kê migration tường minh. Thêm import và class sau InitialIdentity:

```
import { AddAuthEmailOtpGoogle1792713600000 } from
'./migrations/AddAuthEmailOtpGoogle1792713600000';

export default new DataSource(
postgresOptions(
[InitialIdentity1710000000000, AddAuthEmailOtpGoogle1792713600000],
'identity',
),
);
```

Response thêm hai import:

```
import { AddSosFactsAndMissionKind1792713600000 } from
'./migrations/AddSosFactsAndMissionKind1792713600000';
import { AddAiAdvisory1792713600001 } from './migrations/AddAiAdvisory1792713600001';

export default new DataSource(
postgresOptions(
[
InitialResponse1710000000000,
AddRequestSourceIpIndex1710000000001,
AddEvidenceUploadIpQuota1710000000002,
AddSosFactsAndMissionKind1792713600000,
AddAiAdvisory1792713600001,
],
'response',
),
);
```

migrate.ts không cần sửa vì nó gọi runMigrations()/undoLastMigration(). Cần sửa backend/scripts/db/check-migration-down vì script khóa số lượng và thứ tự:

- Identity expected count trước down 1 -> 2; expected sau up cũng là 2.
- Response expected count trước/sau 3 -> 5; down_runs 3 -> 5.
- Thứ tự down Response: run 0 bỏ AI; run 1 bỏ facts/mission kind; run 2 bỏ T5 evidence; run 3 bỏ T4 source-IP; run 4 bỏ InitialResponse.
- Chuyển assertion T5 hiện có từ run 0 sang run 2; assertion T4 từ run 1 sang run 3. Thêm assertion cột/bảng mới đã biến mất sau down tương ứng.
- Checker chỉ chạy sau primary DB reset được phép; không rollback database có dữ liệu.

Cập nhật docs/backend/02-identity-api.md, 03-response-api.md, 07-error-catalog.md và contracts/openapi/identity.json, response.json. Sau này schema dump sau migration phải khớp schema chuẩn; làm theo backend README, không tạo database/Compose test riêng.

12. Thứ tự tự code

1. Chốt contract OTP, Google linking, field SOS, advisory và PUT.
2. Identity migration/schema, rồi DTO/service/controller/provider/config.
3. Cập nhật OpenAPI, error catalog và .env.example.
4. Response migration cho intent/age counts; cập nhật create, approved snapshot, tracking và supplement.
5. PUT với quyền, version, idempotency, immutable event.
6. Coordinator detail/history projection và PII authorization.
7. Worker LLM schema validation, dữ liệu tối thiểu, advisory-only.
8. Rule advisory và UI điều phối; P1–P4 vẫn do coordinator.
9. Delivery mission nối Logistics; giữ capacity guard nếu chưa có capacity model.
10. Cập nhật seed nếu migration mới thêm dữ liệu bắt buộc.
11. Khi tự verify, theo backend README, dùng primary DB/API và loopback ports. Không thêm Compose/database test riêng, không reset nếu thiếu --reset.

13. Checklist tự nghiệm thu sau khi code

Identity

- Đăng ký email trả tiếng Việt và không cấp session trước khi verify.
- OTP sai/hết hạn/dùng lại/resend/rate-limit có error code ổn định.
- Google token sai signature/audience/issuer/expiry/email_verified bị từ chối.
- Google login đã liên kết dùng cùng C48 session flow.
- Email trùng account cũ không tự link.
- WEB giữ HttpOnly cookie/CSRF; NATIVE giữ token response hiện tại.

SOS

- Guest chưa login vẫn tạo SOS được.
- Client cũ không gửi nhu cầu vẫn mặc định rescue.
- Tắt cả rescue và supplies bị từ chối.
- Null nghĩa là chưa biết; 0 nghĩa là đã xác nhận không có.
- LLM không làm chậm trả tracking.
- PUT sai quyền/version/field không được phép không đổi dữ liệu.
- Supplement sau verify chờ người duyệt.

AI, đội và lịch sử

- JSON sai schema, timeout hoặc prompt injection không đổi status/priority/team.
- Coordinator nhìn thấy provenance và lý do AI/rule bằng tiếng Việt.
- Nguy cơ không tự chọn P1–P4.
- Case B=3km bận, C=4km rảnh, D=10km rảnh giải thích được shortlist/thứ tự.
- Coordinator xem được lịch sử; reporter chỉ thấy timeline an toàn.
- Logistics handoff, không phải Response, xác nhận hàng đã giao.

Đây là checklist cho lúc tự triển khai; không có test suite nào được chạy trong lượt tạo PDF này.

Tài liệu gốc

• Scope: docs/c48-project-context.md
• Baseline và Appendix A: docs/c48-technology-and-delivery-plan.md
• Backend task order: docs/backend/README.md
• Schema: docs/backend/schema/identity.sql và docs/backend/schema/response.sql
• Google: backend ID token — https://developers.google.com/identity/sign-in/web/backend-auth, sign-in best practices — https://developers.google.com/identity/siwg/best-practices
• OTP: OWASP email verification — https://cheatsheetseries.owasp.org/cheatsheets/Email_Validation_and_Verification_Cheat_Sheet.html
• LLM: OWASP prompt injection — https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html